Haskell Challenges

ReadMe

Layout: You are given several zip folders named "Level 0", "Level 1", "Level 2" and "Level 3". Inside each of these are one or more pdfs named "Challenge A.pdf", "Challenge B.pdf", "Challenge C.pdf", etc. Each challenge describes a specific function to add to the "Haskell_Challenges.hs" file. The challenges within a level can be completed in any order and each challenge completed earns your team points. You can attempt a challenge many times without penalty and if any team member gets the challenge correct, the entire team earns points. Level 0 is the easiest and the levels get progressively more difficult as you go. You must complete all challenges within a given level to reveal the full password for the next level. Each challenge gives you a piece of the password and putting all pieces together (in order) reveals the full password to use on the zip file for the next level.

Team.data: Make sure your team.data file is sitting next to the "Level" folders that contain the challenges. If this file is not found, you will automatically fail any challenge you attempt since the system won't know who to give credit to.

Challenges: All code should go inside of the Haskell_Challenges.hs file. You should test your code out and call your challenge function several times to make sure it is working correctly. The examples given in each challenge pdf will help. Once you are sure it is working, COMMENT OUT all test code that prints anything to the console. As a check, make sure that running the Haskell_Challenges.scm file produces NOTHING in the terminal. To help you:

- In Notepad++
  - `Ctrl+K` to comment selection (or the line the cursor is on if nothing is selected)
  - `Ctrl+Shift+K` to uncomment selection (or the line the cursor is on if nothing is selected)

Make sure to keep the top lines in the file. This is required to expose your functions to the server code and also to include very important functions for certain challenges. Make sure to save your file and run the following command in Git Bash (or your Mac/Linux terminal) to check your solution:

```
./Check.sh
```

It will ask which level (i.e. 0, 1, 2, or 3) then which challenge letter in that level that you want to check. You might have code from multiple challenges in this same file but you can only check one challenge at a time using this Check.sh script. The Check script will award points if you have successfully passed that challenge. If you fail a challenge, you will need to go back and test your challenge function some more.

Write Your Own Code!: Just like with all other assignments in this course, I may ask you to describe your code. You should know by now that completing any assignment is mostly about figuring out solutions on your own with minimum help from outside sources. When asked, if you cannot adequately explain your solution to a particular challenge, your entire team will not receive credit for that challenge and will not be given a chance to redo it.

Points and Grading: Team points awarded for a challenge are separate (but related to) the actual points you will receive for grading.

Team Points

- Level 0: 1 challenge worth 20 points
- Level 1: 4 challenges worth 20 points each
- Level 2: 4 challenges worth 40 points each
- Level 3: 1 challenge worth 70 points

Course Grade

- Level 0 and all of Level 1 are required for full credit.
    - Each challenge is 6 points for a total of 30 points for the assignment
- Level 2: +4 bonus points each
- Level 3: +6 bonus points each

The team (or teams) with the most team points at the end of class will receive an additional +3 bonus points.